

1 Base R

1. The `starwars` dataset contains the characteristics of 87 different Star Wars characters.

```
glimpse(starwars)
```

```
Rows: 87
```

```
Columns: 14
```

```
$ name      <chr> "Luke Skywalker", "C-3P0", "R2-D2", "Darth Vader", "Leia..."
$ height    <int> 172, 167, 96, 202, 150, 178, 165, 97, 183, 182, 188, 180...
$ mass      <dbl> 77.0, 75.0, 32.0, 136.0, 49.0, 120.0, 75.0, 32.0, 84.0, ...
$ hair_color <chr> "blond", NA, NA, "none", "brown", "brown, grey", "brown"...
$ skin_color <chr> "fair", "gold", "white, blue", "white", "light", "light"...
$ eye_color  <chr> "blue", "yellow", "red", "yellow", "brown", "blue", "blu..."
(plus 8 more variables)
```

What will output from the lines of code given in parts (a)-(d)? **Select ONE each.**

(a) (3 points) `starwars[1,]`

- | | |
|---|--|
| ☐ A vector of length 14 | ☒ A dataframe with 1 row and 14 columns |
| ☐ A vector of length 87 | ☐ A dataframe with 87 rows and 1 column |
| ☐ An error | |

(b) (3 points) `starwars[,4]`

- | | |
|---|--|
| ☐ A vector of length 14 | ☐ A dataframe with 1 row and 14 columns |
| ☐ A vector of length 87 | ☒ A dataframe with 87 rows and 1 column |
| ☐ An error | |

(c) (3 points) `starwars$hair_color`

- | | |
|--|---|
| ☐ A vector of length 14 | ☐ A dataframe with 1 row and 14 columns |
| ☒ A vector of length 87 | ☐ A dataframe with 87 rows and 1 column |
| ☐ An error | |

(d) (3 points) `starwars$4`

- | | |
|---|---|
| ☐ A vector of length 14 | ☐ A dataframe with 1 row and 14 columns |
| ☐ A vector of length 87 | ☐ A dataframe with 87 rows and 1 column |
| ☒ An error | |

(e) (4 points) Some of the values of the `mass` column are NA. Two of the lines of code below returns the average mass (of characters who actually have a mass) and the other two return NA. Briefly explain which are which.

- | | |
|---|---|
| • (A): <code>mean(starwars\$mass != NA)</code> | • (C): <code>mean(starwars\$mass, na.rm = T)</code> |
| • (B): <code>mean(which(!is.na(starwars\$mass)))</code> | • (D): <code>mean(starwars\$mass > 0)</code> |

Solution: Code (A) and (D) will return NA because the `!=` and `<` operators do not work when one or more side is NA. The correct way to check whether an item is NA is with `!is.na()` like in code (B). Or, we can explicitly ignore NA values within the `mean()` function like in (C).

2. Define a character vector `colors` of length 5 and a character vector `names` of length 4.

```
colors <- c("red", "blue", "yellow", "green", "purple")
names <- c("Inky", "Blinky", "Pinky", "Clyde")
```

- (a) (4 points) Identify the length of the vector that will output from the following lines of code, and briefly explain your answer. If the code will instead return an error, explain why.

i. `c(colors, names)`

Solution: The output will be a vector of length 9, since the `c()` command will combine the two vectors into a single one.

ii. `colors + names`

Solution: The output will be an error, because the `+` operator works for numeric objects, and the contents of our vectors are character, not numeric.

iii. `paste(colors, names)`

Solution: The output will have length 5. The `paste` command will connect the character strings together in an element-wise way. So the first elements of each will be pasted together, the second elements of each will be pasted together, etc. The output will be length 5 because that is the longer of the two input vectors.

iv. `paste(colors, "")`

Solution: The output will be a character vector of length 5 and will be exactly the same as the original `colors` vector. The empty string is appended to each individual entry of `colors`.

- (b) (3 points) We would like to identify all of the colors that either start with the letter “g” or are at least five letters long. Which line of R code can be used to do this? **Select ONE.**

- ☐ `(nchar(colors) >= 5) & (substr(colors, 1, 1) == "g")`
☐ `(length(colors) >= 5) & (substr(colors, 1, 1) == "g")`
☒ `(nchar(colors) >= 5) | (substr(colors, 1, 1) == "g")`
☐ `(length(colors) >= 5) | (substr(colors, 1, 1) == "g")`

3. Use the following lines of R code to answer the questions below. Assume that at the beginning, the R environment is empty. Each line will attempt to run once, in order, even if a previous line returned an error.

```
1 rent <- 1200
2 utilities <- 50
3 utilities_adj <- utilities + "10"
4
5 total <- rent + utilities
6 total_adj <- rent + utilities_adj
```

- (a) (3 points) What is the value of the variable `total`? **Select ONE.**

- ☐ 1200
☒ 1250
☐ 1260
☐ undefined

- (b) (3 points) What is the value of the variable `total_adj`? **Select ONE.**

- ☐ 1200
☐ 1250
☐ 1260
☒ undefined

- (c) (5 points) Which line(s) of code will return an error? **Select ALL that apply.**

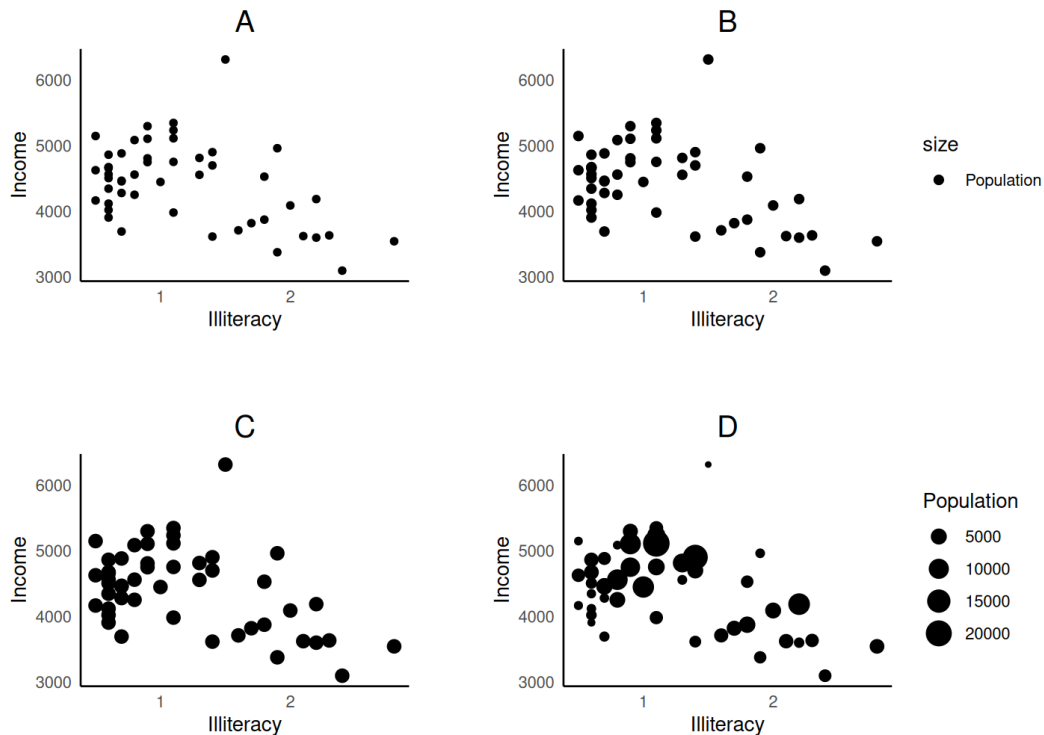
- ☐ 1
☐ 2
☒ 3
☐ 5
☒ 6

- (d) (4 points) Explain why no numbers are printed in the console from running this code.

Solution: We don't expect to see any value print while running this code, because we are only assigning variables with '`<-`' and never actually viewing or printing the value of any of these variables. The value of the variable in the environment will change, but it will not print.

2 ggplot

1. Below are four different `geom_point()` plots of illiteracy rate versus income for US states in the `states` dataframe. Some plots also show the states' population.



- (a) (4 points) Match the four plots (A, B, C, D) to the four different `geom_point()` calls below.

_____ **A** _____ `geom_point(aes(x = Illiteracy, y = Income))`

_____ **D** _____ `geom_point(aes(x = Illiteracy, y = Income, size = Population))`

_____ **B** _____ `geom_point(aes(x = Illiteracy, y = Income, size = "Population"))`

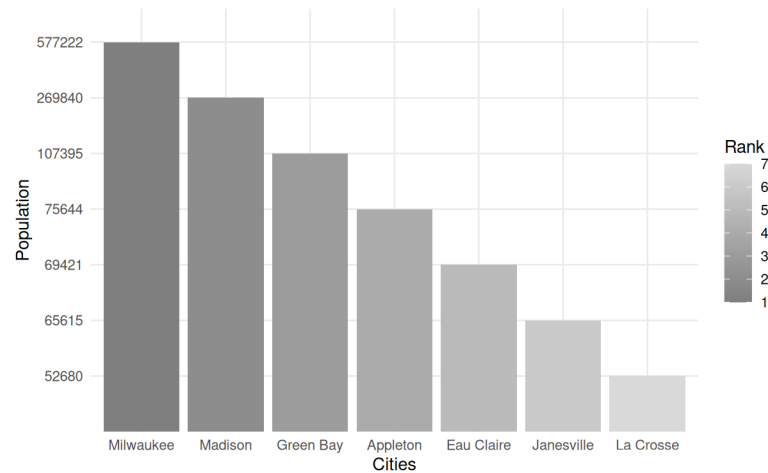
_____ **C** _____ `geom_point(aes(x = Illiteracy, y = Income), size = 3)`

- (b) (3 points) The following code returns an error when trying to run `geom_smooth()`. How should the error be fixed? **Select ONE**.

```
states %>%
  ggplot() +
  geom_point(aes(x = Income, y = Illiteracy)) +
  geom_smooth()
```

- ☐ `geom_smooth()` should be added before `geom_point()`.
- ☐ `x = Income` and `y = Illiteracy` should be moved outside of `aes()`.
- ☒ The `aes()` mapping should be moved inside of `ggplot()`.
- ☐ None of the above; the code should not return any error.

2. The `cities_WI` dataset has 7 rows and 3 columns, containing the name, population, and population rank 1-7 of Wisconsin's seven largest cities. We want to re-create the following bar plot:



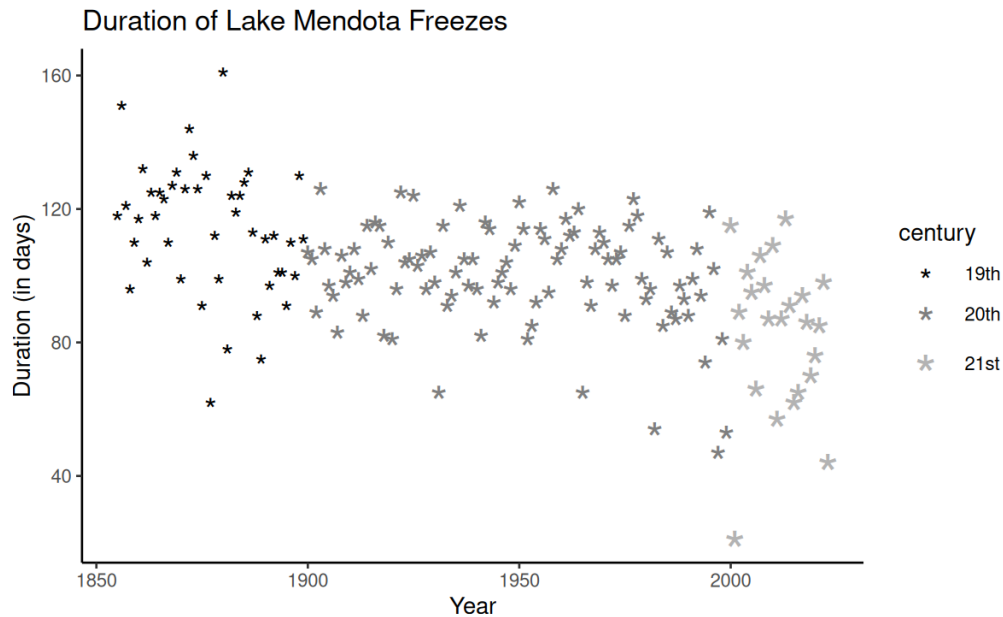
- (a) (3 points) Given that we know the population of the 7 cities, should the plot be created with `geom_bar()` or `geom_col()`? Briefly explain your answer.

Solution: The plot must have been made with `geom_col()`. We use `geom_col()` when we have both an x and y column and can therefore set the height of the bars (population in this case) manually. With `geom_bar()` we only provide one variable and the heights of the bars are calculated automatically based on counts of observations.

- (b) (3 points) How should we write the call to `ggplot()` to make it so the bar interiors are colored based on the cities' Rank? The other plot aesthetics are intentionally left out. **Select ONE.**

- ☒ `ggplot(aes(..., fill = Rank))`
- ☐ `ggplot(aes(..., col = Rank))`
- ☐ `ggplot(aes(...), fill = Rank)`
- ☐ `ggplot(aes(...), col = Rank)`

3. The scatter plot below was created with the mendota data from lecture. It shows the duration of Lake Mendota freezes over time, and also indicates the century each point belongs to.



- (a) (5 points) For each aesthetic used in the plot, indicate whether it is a variable or constant aesthetic, or if it is not used in the plot. **Select ONE each.**

x :	☒ Variable	☐ Constant
y :	☒ Variable	☐ Constant
color :	☒ Variable	☐ Constant
shape :	☐ Variable	☒ Constant
size :	☒ Variable	☐ Constant

- (b) (3 points) Which two aesthetics correspond to the same variable in the data? What information is being conveyed by those two aesthetics?

Solution: color and size are both being used to refer to the century each data point belongs to.

- (c) (3 points) Which line of code below create a red line parallel to the x-axis representing the mean duration? **Select ONE.**

☒ `geom_hline(yintercept = mean(mendota$duration), color = "red")`
☐ `geom_vline(yintercept = mean(duration), color = "red")`
☐ `geom_line(yintercept = mean(mendota$duration), color = "red")`
☐ `geom_hline(yintercept = mean(duration), color = "red")`
☐ `geom_vline(xintercept = mean(mendota$duration), color = red)`
☐ `geom_line(xintercept = mean(duration), color = red)`

4. The fictitious dataset `state_employees` contains columns for the ID, department, number of years of service, and annual salary for 100000 state employees.

```
state_employees %>%
  sample_n(5)
```

	employee_id	department	years_service	salary
	<int>	<chr>	<dbl>	<dbl>
1	10783	agriculture	3	23660
2	69923	public_safety	7	33917
3	35260	corrections	8	28847
4	3336	agriculture	4	25772
5	45582	social_services	5	26632

The graph below shows the average annual salary for the five departments in the dataset.



- (a) (6 points) In the plot above, which aesthetics are variable aesthetics? **Select ALL that apply.**

☒ x

☐ fill

☒ y

☒ text

☐ color

☐ line

- (b) (3 points) Was the plot above made using `geom_bar()` or `geom_col()`? Briefly explain how you can tell.

Solution: This plot must have been made with `geom_col()` after calculating the average annual salary since we need to provide both an x and a y to specify the height of the bars. `geom_bar()` would give a count of each department which is not what we want in this case.

5. Your zoologist friend is studying three species of zebra. She has gathered data on the physical characteristics of 160 of these zebras and asks for your help analyzing it. Below are 5 example rows from full dataset.

	length	height	sex	species
	<i><dbl></i>	<i><dbl></i>	<i><chr></i>	<i><chr></i>
1	237	142	F	mountain
2	245	145	M	plains
3	216	128	F	mountain
4	235	139	F	plains
5	282	152	M	grevy's

She sends you the following message about how she wants to graph the data: “I want to make a scatter plot of each zebra’s body length and shoulder height, with a single smooth line showing the trend. I want the color of the points to be different for the different species, and I want all of the points to be slightly transparent in case they overlap”.

- (a) (6 points) Which ggplot2 graph types should you add to the plot to fulfill her request? **Select ALL that apply.**

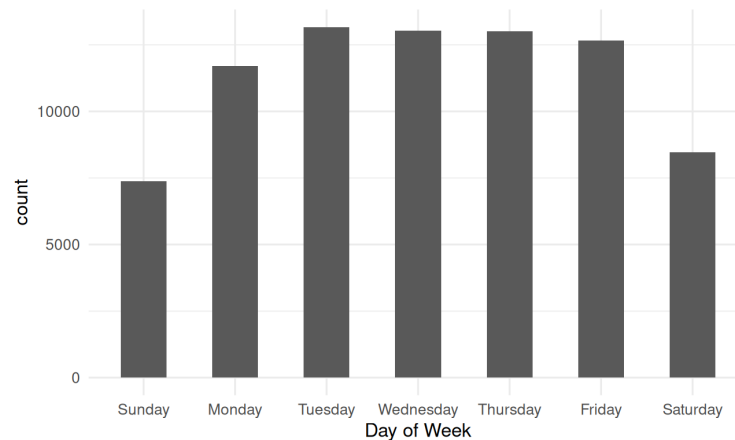
- | | |
|---|--|
| ☐ geom_density() | ☒ geom_smooth() |
| ☒ geom_point() | ☐ geom_histogram() |
| ☐ geom_line() | ☐ geom_boxplot() |

- (b) (4 points) For each of the following aesthetics in your plot, identify whether they would need to be global variable, local variable, or constant. **Select ONE each.**

- | | | | |
|---------|---|--|---|
| x : | ☒ Global | ☐ Local | ☐ Constant |
| y : | ☒ Global | ☐ Local | ☐ Constant |
| color : | ☐ Global | ☒ Local | ☐ Constant |
| alpha : | ☐ Global | ☐ Local | ☒ Constant |

6. The dataframe `births` (with 79,421 rows and 1 column) records the day of the week for 79,421 births. The code below (some details omitted) creates a graph to count the number of births for each day.

```
births %>%
  group_by(day_of_week) %>%
  summarize(count = n()) %>%
  ggplot(aes(x = day_of_week, y = count)) +
    geom_col()
```



- (a) (3 points) Explain why we need to create a new variable called `count`. Identify a different geom that could produce the same plot without needing to tidy the data or manually create this new variable.

Solution: Since `geom_col()` requires both an `x` and a `y` variable, we need to pre-calculate the counts so that we can set the bar height manually. We could instead use `geom_bar()` which only takes a categorical `x` variable and automatically computes the counts.

- (b) (4 points) You want to make the following changes to the plot: Add labels at the top of each bar displaying the count for that day, and add a line showing the count of the tallest bar. Which geoms do you need to add to your plot? **Select ALL that apply.**

☐ `geom_point()`

☒ `geom_hline()`

☒ `geom_text()`

☐ `geom_vline()`

- (c) (3 points) You want to color the bar areas pastel blue. Which aesthetics do you need to add to your plot? **Select ONE.**

☐ `color` as a variable aesthetic

☐ `fill` as a variable aesthetic

☐ `color` as a constant aesthetic

☒ `fill` as a constant aesthetic

3 dplyr

1. The `birthwt` dataset records birth weight and risk factors for 189 pregnancies. We are interested in columns `low` (0 or 1, indicating low birth weight), `smoke` (0 or 1, indicating mother's smoking status), and `bwt` (birth weight of newborn in grams).

```
> birthwt %>% select(low, smoke, bwt) %>% glimpse()
Rows: 189
Columns: 3
$ low    <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
$ smoke  <int> 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, ...
$ bwt    <int> 2523, 2551, 2557, 2594, 2600, 2622, 2637, 2637, 2663, 2665, 2722, ...
```

Use these function names as well as the column names to fill in the blanks below. You might not need to use every function/column, and you might need to use some more than once.

- | | | |
|-------------------------|--------------------------|--------------------------|
| • <code>select</code> | • <code>summarize</code> | • <code>slice_min</code> |
| • <code>filter</code> | • <code>count</code> | • <code>median</code> |
| • <code>group_by</code> | • <code>min</code> | • <code>mean</code> |

- (a) (3 points) Fill in the blanks to count the number of rows for every combination of low birth weight (low vs normal) and mother's smoking status (yes vs no).

```
birthwt %>%
  group_by(low, smoke) %>%
  summarize(numRows = n())
```

- (b) (5 points) Fill in the blanks to find the average and median weight of the 50 smallest newborns in the data.

```
birthwt %>%
  slice_min(bwt, n = 50) %>%
  summarize(avgWt = mean(bwt), medWt = median(bwt))
```

2. The cabbages dataset gives the cultivar type, date planted, head weight (in kg) and vitamin C content of 60 cabbages.

```
> glimpse(cabbages)
Rows: 60
Columns: 4
$ Cult   <fct> c39, c39, c39, c39, c39, c39, c39, c39, c39, c39, c39, c39, c39, c39, ...
$ Date   <fct> d16, d16, d16, d16, d16, d16, d16, d16, d16, d16, d20, d20, d20, d20, ...
$ HeadWt <dbl> 2.5, 2.2, 3.1, 4.3, 2.5, 4.3, 3.8, 4.3, 1.7, 3.1, 3.0, 2.8, 2.8, 2.7, ...
$ VitC   <int> 51, 55, 45, 42, 53, 50, 50, 52, 56, 49, 65, 52, 41, 51, 41, 45, 51, 45, ...
```

Use these function names to fill in the blanks below. You might not need to use every function, and you might need to use some more than once.

- | | | |
|------------|-------------|---------|
| • relocate | • summarize | • n |
| • select | • max | • count |
| • group_by | • slice_max | • mean |

- (a) (3 points) Fill in the blanks to remove the VitC column, then print the largest cabbage for each of the cultivar types.

```
cabbages %>%
  select(-VitC) %>%
  group_by(Cult) %>%
  slice_max(HeadWt)
```

- (b) (4 points) Fill in the blanks to count the number of cabbages for each cultivar-date combination, and calculate the average vitamin C content for each cultivar-date combination.

```
cabbages %>%
  group_by(Cult, Date) %>%
  summarize(counts = n()),
  avgVitC = mean(VitC))
```

3. Your zoologist friend is studying three species of zebra. She has gathered data on the physical characteristics of 160 of these zebras and asks for your help analyzing it. Below are 5 example rows from full dataset.

	length	height	sex	species
	<dbl>	<dbl>	<chr>	<chr>
1	237	142	F	mountain
2	245	145	M	plains
3	216	128	F	mountain
4	235	139	F	plains
5	282	152	M	grevy's

- (a) (3 points) You want to re-structure the data so that all of the measurements are in a single column called *measurement*, with a column called *type* labeling what is being measured.

	sex	species	type	measurement
	<chr>	<chr>	<chr>	<dbl>
1	M	grevy's	length	273
2	M	grevy's	height	146
3	M	grevy's	length	294
4	M	grevy's	height	157
5	M	grevy's	length	279
6	M	grevy's	height	157

Which line of R code should be used? **Select ONE.**

- ☒ `pivot_longer(length:height, names_to = "type", values_to = "measurement")`
- ☐ `pivot_wider(length:height, names_to = "type", values_to = "measurement")`
- ☐ `pivot_longer(names_from = "type", values_from = "measurement")`
- ☐ `pivot_wider(names_from = "type", values_from = "measurement")`

- (b) (4 points) Consider the following code which creates a new dataframe, `zebra_summary`.

```
zebra_summary <- zebras %>%
  group_by(species) %>%
  summarize(avgLength = mean(length),
            avgHeight = mean(height))
```

Which of the following statments are true about `zebra_summary`? **Select ALL that apply.**

- ☒ `zebra_summary` has three rows.
- ☐ `zebra_summary` has two columns.
- ☒ `zebra_summary` has a column called "species".
- ☐ `zebra_summary` has a column called "sex".

4. A dataframe `lions` has 32 rows, each representing a unique lion.

It has two numeric columns: `age`, each lion's age in decimal years, and `proportion.black`, the percentage of that lion's nose which is black. It has a logical column `adult`, which is `TRUE` if `age` is greater than 3 and `FALSE` otherwise.

- (a) (4 points) Consider the following code which creates a new dataframe, `lions_summary`.

```
lions_summary <- lions %>%  
  group_by(adult) %>%  
  summarize(averageAge = mean(age),  
            averagePropBlack = mean(proportion.black))
```

Which of the following statements are true about `lions_summary`? **Select ALL that apply.**

- ☒ `lions_summary` has 2 rows.
 - ☐ `lions_summary` has 2 columns.
 - ☒ `lions_summary` has a column called "adult".
 - ☐ If there were any NA values in the `age` column of `lions`, all the values of `averageAge` in `lions_summary` will be NA.
- (b) (6 points) Which lines of code can be used to keep only the rows of `lions` whose age is greater than 3? **Select ALL that apply.**

- ☐ `lion %>% select(adult)`
- ☐ `lion %>% select(age)`
- ☐ `lion %>% select(age > 3)`
- ☒ `lion %>% filter(adult)`
- ☐ `lion %>% filter(age)`
- ☒ `lion %>% filter(age > 3)`

- (c) (3 points) The dataframe `lion_traits` has 25 rows, each representing a unique lion. It has three columns: `id`, a number that uniquely identifies each lion, `sex`, a categorical variable for whether the lion is male or female, and `length`, the length of the lion's body in meters.

How many rows and columns will result from the following join operation? Briefly explain your answers. If the code returns an error, explain why.

```
left_join(lions, lion_traits)
```

Solution: The code will return an error. There are no columns in common between the two datasets, so there is nothing to join by.

5. The fictitious dataset `state_employees` contains columns for the ID, department, number of years of service, and annual salary for 100000 state employees in five departments.

```
state_employees %>%
  sample_n(5)
```

	employee_id	department	years_service	salary
	<int>	<chr>	<dbl>	<dbl>
1	10783	agriculture	3	23660
2	69923	public_safety	7	33917
3	35260	corrections	8	28847
4	3336	agriculture	4	25772
5	45582	social_services	5	26632

- (a) (4 points) Consider the following code which creates a new dataframe, `employee_summary`.

```
employee_summary <- state_employees %>%
  filter(department != "agriculture") %>%
  group_by(department) %>%
  summarize(avg_salary = mean(salary), employees = n())
```

Which of the following statements are TRUE about `employee_summary`? **Select ALL that apply.**

- ☒ `employee_summary` has 4 rows.
 - ☐ `employee_summary` has 5 rows.
 - ☒ `employee_summary` has a column called "department".
 - ☐ If were any NA values in the salary column of the original data, all the values of `avg_salary` in `employee_summary` will be NA.
- (b) (4 points) Continue working with the 100000 rows of employee salary data. Another dataset called `employment_type` has 74033 rows and two columns: numeric `employee_id` and categorical `type` which labels each employee as either full-time or part-time. The 74033 employee IDs are all also part of the larger `state_employees` dataset.

How many rows and columns result from following join? Which columns will contain NA values?

```
left_join(state_employees, employment_type)
```

Solution: The joined data will have 100000 rows, just like the original `state_employees` data. There will be 5 columns: the four original columns from `state_employees` and the column for `employment_type` from the `employment_type` dataframe.

This `type` column will have NA values for the rows that appear in `state_employees` but not in `employment_type`.

6. (5 points) Researchers perform two experiments. In the first experiment, researchers record the participants' names and 8 other variables, which are stored in `df1`. In the second experiment, researchers record participants' names and 6 other variables, which are stored in `df2`. The name column is the only variable that appears in both dataframes.

`df1` has 15 rows and `df2` has 25 rows (one row per participant). 5 people participated in both experiments, so they appear in both dataframes.

Give the number of rows and the number of columns that result from the following join operations.

Join	Number of rows	Number of columns
<code>inner_join(df1, df2)</code>	<u>5</u>	<u>15</u>
<code>left_join(df1, df2)</code>	<u>15</u>	<u>15</u>
<code>right_join(df1, df2)</code>	<u>25</u>	<u>15</u>
<code>full_join(df1, df2)</code>	<u>35</u>	<u>15</u>
<code>anti_join(df1, df2)</code>	<u>10</u>	<u>9</u>

7. (5 points) A company is creating a video game where the player fights increasingly difficult monsters through different layers of a dungeon. One team creates a dataset `monster_stats` with the combat statistics of different monsters. The column `name` gives the monster's name, `level` gives the difficulty level of the monster, and `atk` gives the monster's attack.

	name <chr>	level <dbl>	atk <dbl>
1	skeleton	1	30
2	ooze	2	45
3	haunt	4	75

A different team creates a dataset `monster_locations` that describes where different monsters appear. The column `name` gives the monster's name, `level` gives the layer of the dungeon where it lives, and `enviro` gives the monster's preferred environment.

	name <chr>	level <dbl>	enviro <chr>
1	chicken	0	mild
2	spider	1	humid
3	skeleton	1	arid
4	ooze	3	humid

The following R code is used to join the two dataframes based on the `name` column.

```
full_join(monster_stats, monster_locations, join_by(name))
```

Using the grid below, re-create the dataframe that would result from the call to `full_join()`. Use the top shaded row for the column names and the un-shaded cells for the data values. Be sure to account for any missing data. The column names and the ordering of rows/columns do not need to be exact to get full credit, but the contents of the data must be accurate.

	name	level.x	level.y	atk	enviro
1	skeleton	1	1	30	arid
2	ooze	2	3	45	humid
3	haunt	4	NA	75	NA
4	chicken	NA	0	NA	mild
5	spider	NA	1	NA	humid

8. The dataset `students` gives the name, lecture section, homework, quiz, and exam grades of several students in a class. Below are 5 example rows from the full dataset.

	name	section	hw	quiz	exam
	<i><chr></i>	<i><dbl></i>	<i><dbl></i>	<i><dbl></i>	<i><dbl></i>
1	John	1	90	95	85
2	Arya	1	87	92	83
3	Fred	2	93	99	84
4	Riley	2	85	89	75
5	Tyler	2	67	70	61

- (a) (4 points) We want to make a new column that combines `hw`, `quiz`, and `exam` into an overall final score, then converts that number into a categorical letter grade based on a scale.

For example, a student who scores between 92-100 should get an "A", a student who scores between 88-92 should get an "AB", and so on. Which `dplyr` commands should we use to accomplish this goal? **Select ALL that apply.**

☒ `mutate()`

☐ `relocate()`

☐ `as.factor()`

☒ `case_when()`

- (b) (3 points) Starting from the original `students` data, we want to create a dataframe that has a column for "assignment type" and a column for "value" rather than three separate `hw`, `quiz`, `exam` columns. Which R code can be used to create this data?. **Select ONE.**

	name	section	assign_type	value
	<i><chr></i>	<i><dbl></i>	<i><chr></i>	<i><dbl></i>
1	John	1	hw	90
2	John	1	quiz	95
3	John	1	exam	85
4	Arya	1	hw	87

☐ `students %>% summarize(assign_type = c(hw, quiz, exam), value = n())`

☐ `students %>% pivot_wider(names_from = assign_type, values_from = value)`

☒ `students %>% pivot_longer(c(hw, quiz, exam), names_to = assign_type)`

☐ `students %>% count(c(hw, quiz, exam))`

9. (5 points) In the dataframe `cakes`, a bakery records the flavor and occasion for each cake order they receive each week. For example, this week they received 4 orders for chocolate birthday cakes.

```
> cakes
# A tibble: 8 × 3
  flavor    occasion num_ordered
<chr>    <chr>         <dbl>
1 chocolate birthday         4
2 vanilla  birthday         3
3 vanilla  wedding            1
4 redvelvet wedding            1
5 chocolate wedding            2
6 carrot   birthday            2
7 vanilla  other                 4
8 redvelvet other                 2
```

The bakery wants to structure the data such that there is a separate column for each occasion type, rather than a single “occasion” column.

Using the grid below, re-create the dataframe that would result from the call to `pivot_wider`. Use the top shaded row for the column names and the un-shaded cells for the data values. Be sure to account for any missing data.

```
pivot_wider(cakes, names_from = "occasion", values_from = "num_ordered")
```

	flavor	birthday	wedding	other
1	chocolate	4	2	NA
2	vanilla	3	1	4
3	redvelvet	NA	1	2
4	carrot	2	NA	NA